MPI Benchmark

Hardware specifications

All benchmarks were run on a 2018 MacBook Pro running the Sonoma 14.6.1 operating system with a single 2.3 GHz Quad-Core Intel i5 processor. This processor has 4 physical cores, and 8 logical cores, as Hyper-Threading is enabled. Each physical core has access to two L1 caches of 32 KB each, one for instructions and one for data. Additionally, each core has a 256 KB L2 cache, while all cores share a 6 MB L3 cache.

The device has 16 GB of LPDDR3 RAM spread across two equally sized banks. Additionally, the memory has a clock speed of 2133 MHz and does not support ECC (error-correcting code).

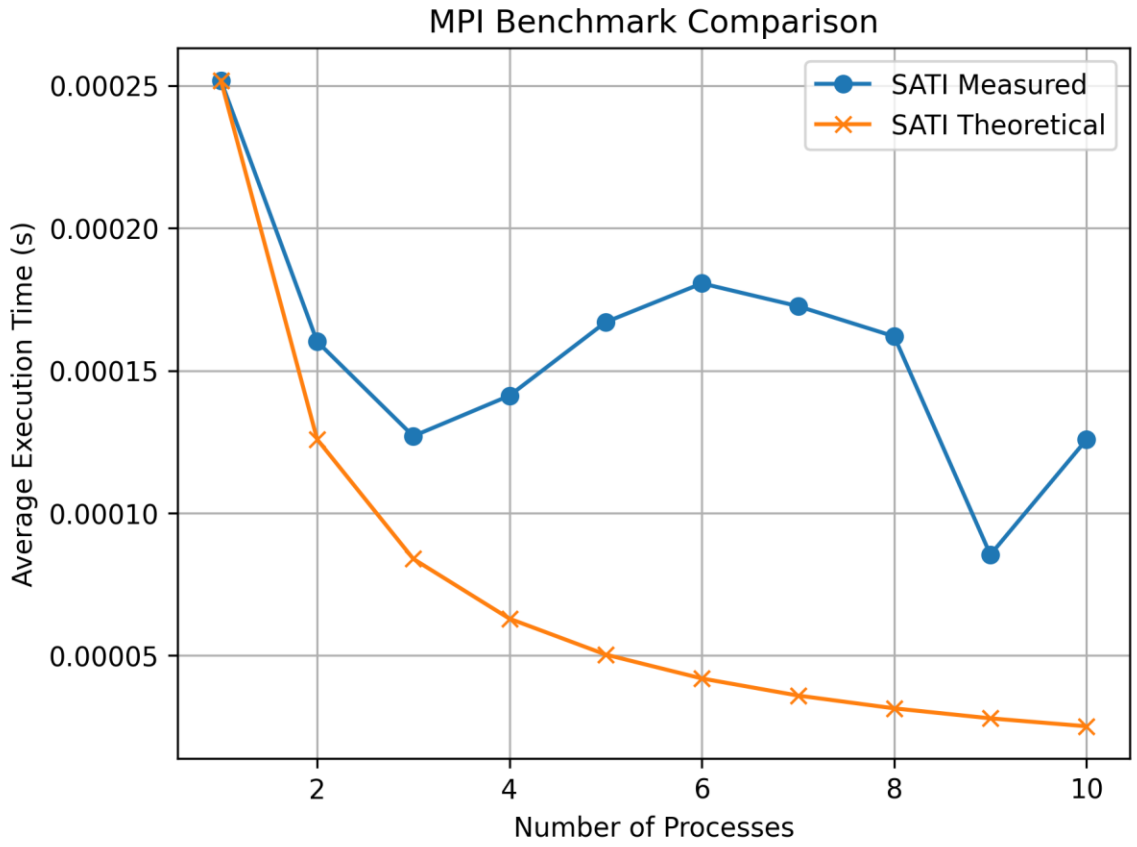
Hardware Environment

In order to provide reliable results, various methods were used to provide consistency across program runs. All benchmarks were performed with the laptop plugged in and fully charged, with energy-saving mode disabled. Additionally, all benchmarks were performed with all other apps closed and a CPU temperature of at most 60° C. Furthermore, the '*sudo renice -n -20 -p $$*' command was executed in the terminal before launching all benchmarks. This sets the scheduling priority of the terminal to maximum, potentially reducing the amount of CPU time used by other processes during benchmarking.

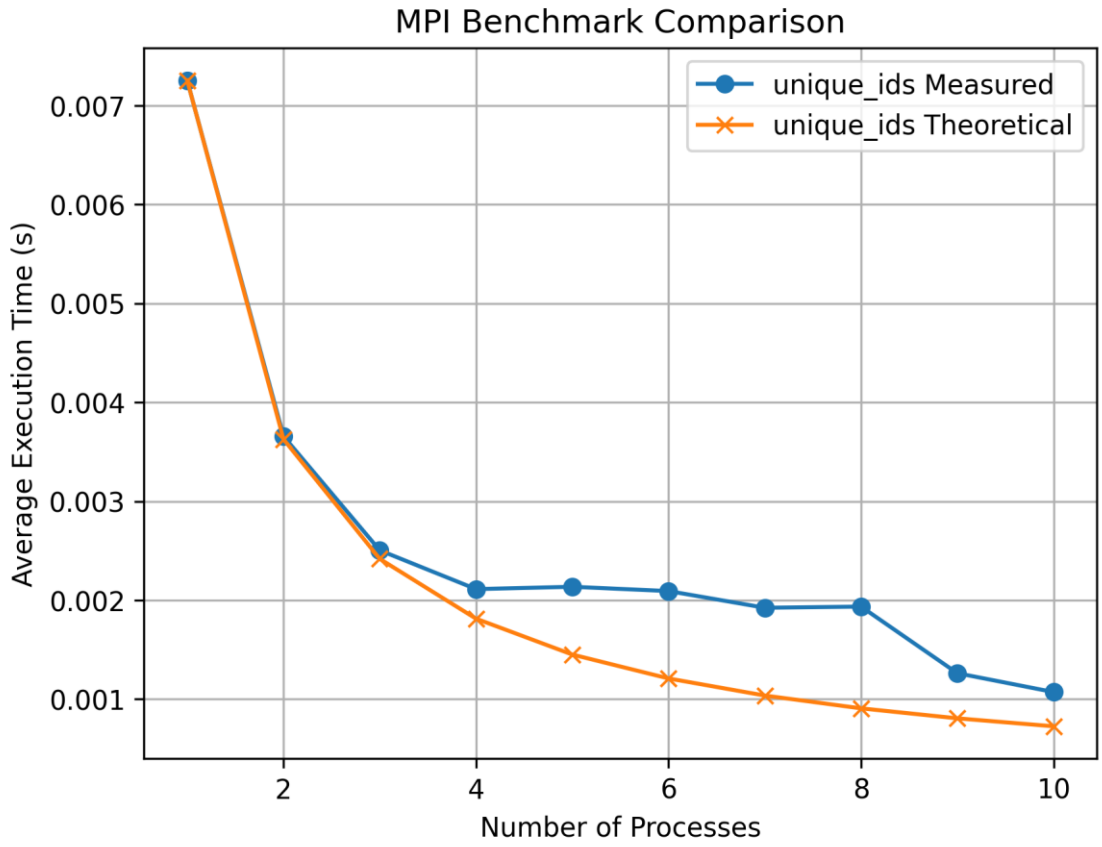
MPI Environment

All source code was compiled and executed using Open MPI version 5.0.8, using Apple Clang 16.0.0 as the compiler. Additionally, various flags were used to ensure consistent multi-processing performance. The `*--map-by core*` flag was used to ensure processes are assigned to their own physical cores when available, and then one per logical core between 5 and 8 processes. When more than 8 processes are used during execution, shared processes on logical cores are unavoidable. Additionally, the `*--bind-to core*` flag ensures processes are not moved between cores during execution. Furthermore, *O3* optimization was used when compiling all source code.

Satisfiability Problem



Unique ID Numbers

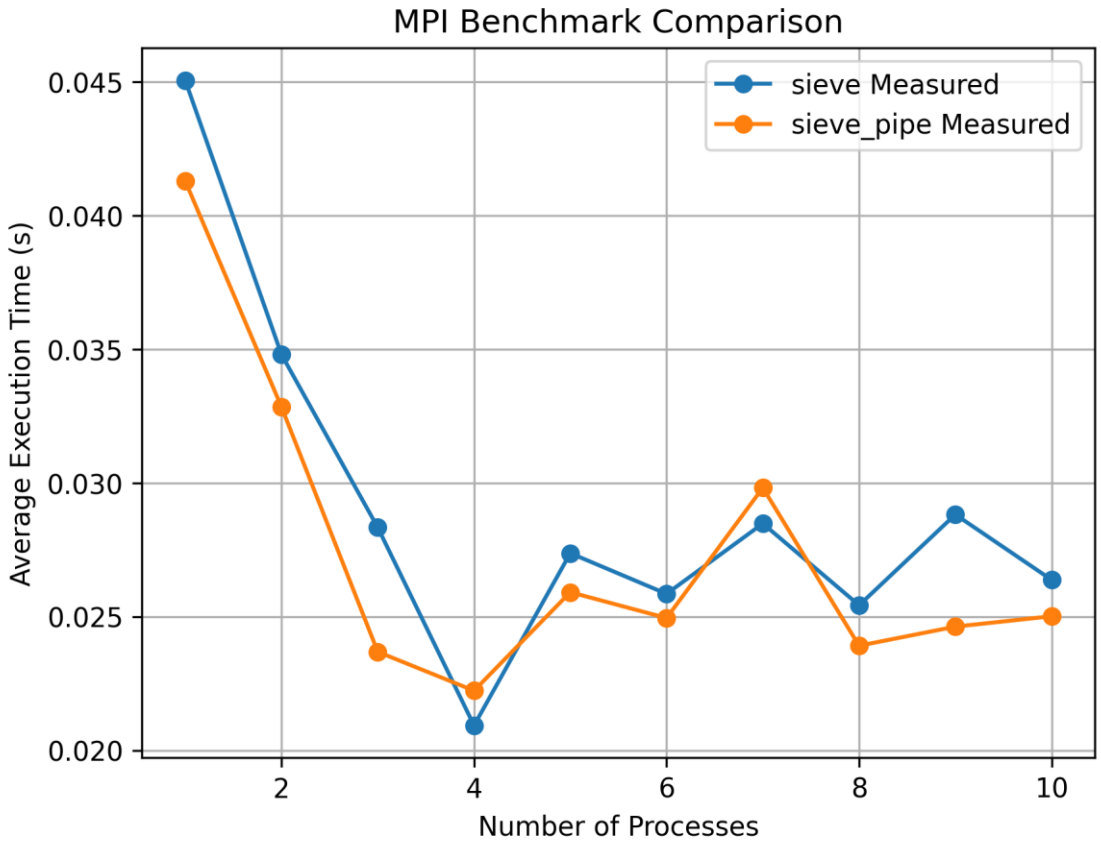


Prime Number Sieve

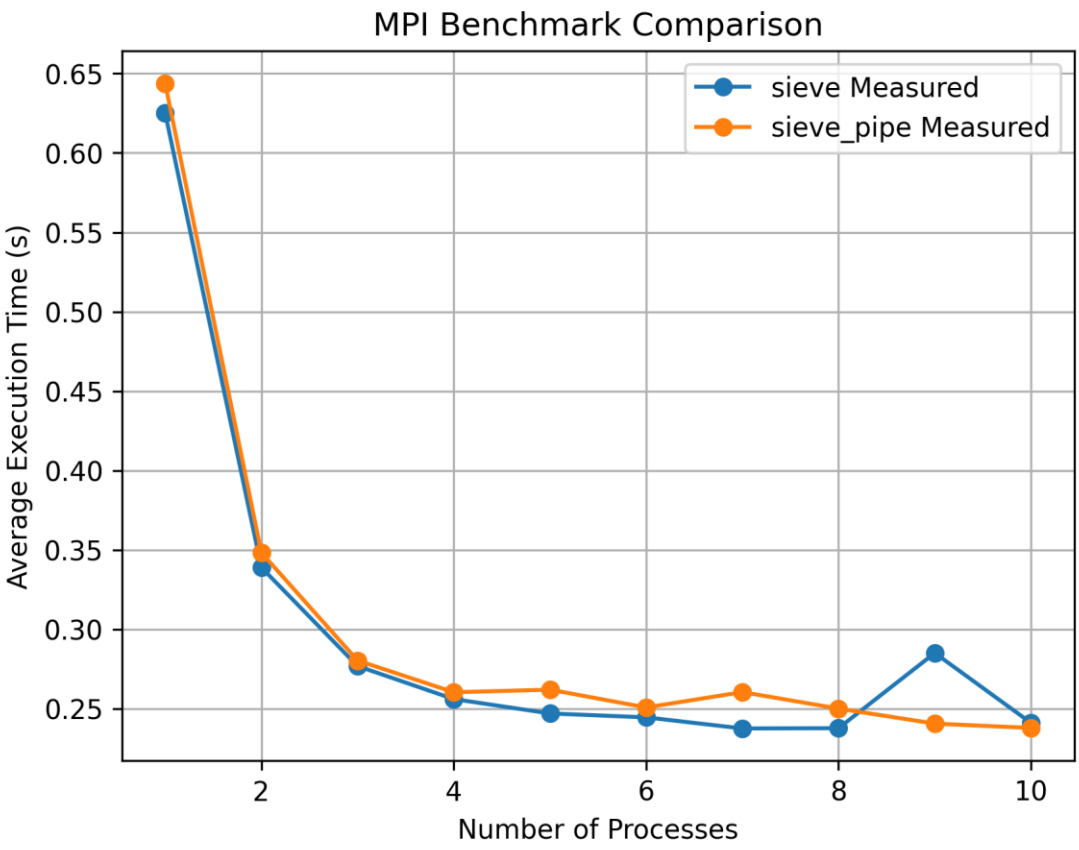
The Sieve algorithm, taking *n* as input, determines the number of primes less than n. The original source code we provided divides numbers from 0 to n between processes. When a prime is discovered, all of its multiples up to *n* are marked as composites, meaning they are not primes. The next unmarked number can be confirmed as a prime, and is transmitted to all processes. Processes can then mark its multiples in their section of the numbers from *0* to *n*.

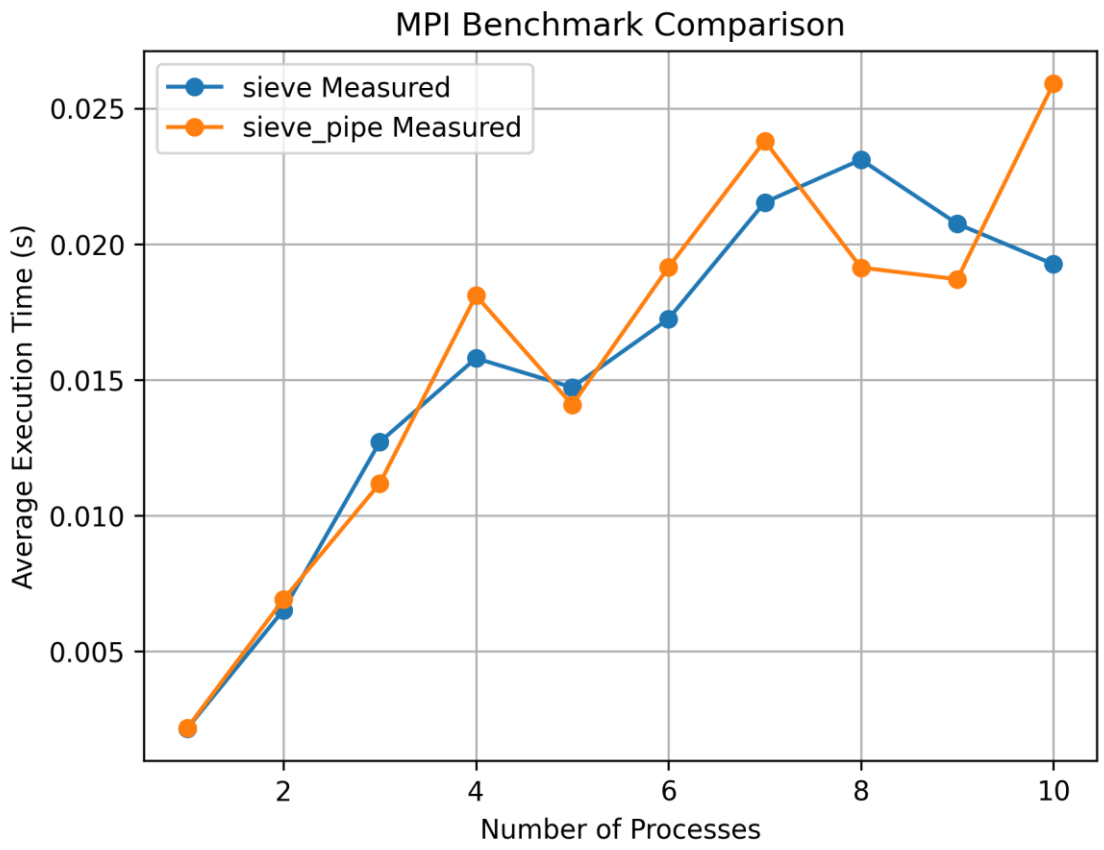
Two different methods were used to communicate found primes across processes. In the original provided code, when a prime is discovered, it broadcasts found primes to all other processes. Other processes can then calculate multiples, and again broadcast found primes. In the modified version, instead of a global broadcast, primes are transmitted one by one, from one process to the next. This forms a communication pipe, allowing all processes to receive found primes from lower processes without every broadcasting globally.

When benchmarked with finding the number of primes below *10,000,000,* the pipe program generally achieved slightly better performance over the broadcast program. Additionally, both programs achieved maximum performance when using 4 processes, matching the number of physical cores on the hardware. Performance degraded slightly, but remained relatively equal when using 3 to 10 processes. However, with less than 3 processes, performance degraded significantly. Interestingly, the broadcast version was consistently able to achieve slightly faster execution with 4 processes. However, the pipe method achieved slightly better performance with other process amounts. Benchmark results when *n* = *10,000,000* can be seen below.



When different numbers for *n* were used, results altered significantly. Seen below is a benchmark where *n* = *100,000,000*. In this scenario, 8 processors were ideal, with the broadcast version achieving slightly better performance than linear communication. However, in the next plot, where *n* was set to *1,000,000*, performance was best when only a single processor was used. This is likely because *n* needs to be large for the communication overhead to outweigh the advantages of parallel computations.


MPI Benchmark Comparison
sieve Measured
sieve_pipe Measured
Average Execution Time (s)
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0.30
0.25
2
4
6
8
10
Number of Processes



Conway's Game of Life